

- Secrets
- ABAP
- Ansible
- Apex
- AzureResourceManager
- C
- C#
- C++
- CloudFormation
- COBOL
- CSS
- Dart**
- Docker
- Flex
- Go
- HTML
- Java
- JavaScript
- JCL
- Kotlin
- Kubernetes
- Objective C
- PHP
- PL/I
- PL/SQL
- Python
- RPG
- Ruby
- Scala
- Swift
- Terraform
- Text
- TypeScript
- T-SQL
- VB.NET
- VB6
- XML



## Dart static code analysis

Unique rules to write Clean DART Code

All rules **115** Bug **15** Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name...

"void" variables should not be assigned a value

Bug

Constant patterns should not be used with type literals

Bug

"is!" should be used instead of "Is"

Bug

Getters should not be recursive

Bug

Nullable type parameter values should not be null checked with '!

Bug

"const" modifier should not be redundant

Bug

Setters should not declare return types

Bug

Inappropriate collection calls should not be made

Bug

Unnecessary equality checks should not be made

Bug

The original exception object should be rethrown

Bug

Literal constructors parameters of @immutable classes should be const

Code Smell

Unnecessary widget containers should be removed

## "void" variables should not be assigned a value

Analyze your code

Intentionality - Logical

Reliability

Bug Major

Variables of type `void` should not be assigned a value.

Why is this an issue?

How can I fix it?

More Info

Unlike other languages where `void` is the absence of a type (and related value), `void` in Dart is a type, which is not compatible with any other type, since it is conceptually a supertype of `Object`.

Therefore, in Dart it's possible to write the following code:

```
void x = 3; // A variable of type void can be assigned a value of type int
void f(void x) => x; // A function that takes a void input argument and returns it as output
```

The variable `x` declared above, and the result of `f(x)` are effectively of type `void`. However, while `void` variables can be declared, initialized and even computed, they cannot be used by any function that expects a non-void type, by the very definition of `void` type.

```
print(x); // Error: This expression has type 'void' and can't be used.
print(f(x)); // Error: This expression has type 'void' and can't be used.
```

Therefore, assigning a value to a variable of type `void` is almost certainly a mistake.

For example, in the cases where the type is not known, the variable should have not been declared as `void`, but as `dynamic`:

- `void` variables can be assigned values of any type, but cannot be assigned to any non-`void` variable
- `dynamic` variables, like `void` ones, can be assigned values of any type, and, unlike `void` variables, can be assigned to any non-`dynamic` variable

Notice that, because `void` is a type, it can be used as a valid type parameter for a generic class, and that can lead to indirect or "hidden" `void` assignments:

```
class Wrapper<T> {
  T value;
  Wrapper(this.value);
}

void main() {
  Wrapper<void> w = Wrapper(3); // This is a "hidden" assignment to void
  w.value = 3; // This is an explicit assignment to a field of type void
}
```

Another example of a "hidden" assignment to `void` is when assigning generic unconstrained type parameters in generic methods:

```
void f<T>(T x, T y) {
  y = x; // This is a "hidden" assignment to void
  T z;
  z = x; // This too, is a "hidden" assignment to void
}
```

However, such scenarios are not reported by the rule because for any non-`void` type `T`, the assignment is valid. On the other hand, calling the function `f` with anything other than `void` parameters will be reported as an issue:

```
f(42, 'a string'); // Both 1st and 2nd parameters are non compliant
```

Available In:

sonarcloud

